

Data Engineering Intern Project: Local Data Pipeline

Project Overview

Your objective is to build a local, containerized Data Engineering environment using **Docker Compose** and **Apache Airflow**. Once the infrastructure is running, you will write a data pipeline (DAG) to extract, clean, and load a messy dataset of financial transactions into a local PostgreSQL database.

This project simulates a real-world "Bronze to Silver" data architecture.

Phase 1: Infrastructure Setup

You must define your infrastructure as code using Docker.

Requirements:

1. Create a `docker-compose.yml` file that spins up the following services:
 - **PostgreSQL:** This will act as our local Data Warehouse. Create a database called `analytics_db`.
 - **Airflow Webserver:** The UI for monitoring our DAGs (mapped to port `8080`).
 - **Airflow Scheduler:** The engine that triggers the tasks.
2. **Configuration:** Configure Airflow to use the `LocalExecutor` so you do not need to spin up separate worker containers. Search also for the other types of executors and what they are.

Phase 2: The Raw Data

Create a file named `raw_transactions.csv` in your project folder and paste the following messy data into it. Notice the inconsistent date formats, missing currencies, and messy text cases.

Code snippet:

```
transaction_id,txn_date,amount,currency,status,merchant
TXN-1001,12-05-2026,$45.50,USD,SUCCESS,Amazon
TXN-1002,2026/05/13,12.00,,failed,STARBUCKS
TXN-1003,05-14-26,-8.99,EUR,Success,Netflix
TXN-1004,,105.25,USD,PENDING,Apple
TXN-1005,16-05-2026,0.00,,SUCCESS,Test_Transaction
```

Phase 3: The Airflow Pipeline (DAG)

Write a Python file (`transaction_pipeline.py`) that defines an Airflow DAG to process this CSV. Your DAG should run daily and consist of three distinct tasks:

Task 1: Extract (`extract_data`)

- Read the `raw_transactions.csv` file.
- Load the raw, unedited data directly into a table in your Postgres database called `bronze_transactions`.
- *Hint: You can use pandas or polars.*

Task 2: Transform & Clean (`clean_data`)

- Read the data from the `bronze_transactions` table.
- **Cleaning Requirements:**
 1. Standardize the `txn_date` column to a single standard format (e.g., `YYYY-MM-DD`). Drop any rows where the date is entirely missing.
 2. Remove the \$ sign from the `amount` column and ensure it is cast as a numeric float.
 3. Fill missing values in the `currency` column with a default value of `'USD'`.
 4. Standardize the `status` column to be entirely uppercase (e.g., `'SUCCESS'`, `'FAILED'`).

Task 3: Load (`load_silver`)

- Take the cleaned dataframe and load it into a final Postgres table called `silver_transactions`.
- Ensure this task only runs if the `clean_data` task succeeds.